

BlockingCollection in C# with Examples

In this article, I am going to discuss the **BlockingCollection in C#** with Examples. Please read our previous article where we discussed [ConcurrentBag Collection Class in C#](#) with Examples. BlockingCollection Class in C# is a Thread-Safe Collection Class. It is an implementation of the Producer-Consumer Pattern. It provides features bounding and blocking to support Producer-Consumer Pattern. It is only the Concurrent Collection class that supports the Bounding and Blocking features. At the end of this article, you will understand the following pointers.

1. [What is BlockingCollection in C#?](#)
2. [How to Create a BlockingCollection<T> instance in C#?](#)
3. [How to Add Elements into a BlockingCollection<T> in C#?](#)
4. [How to access a BlockingCollection in C#?](#)
5. [Initializing BlockingCollection in C# using Collection Initializer](#)
6. [How to Remove Elements from BlockingCollection<T> Collection in C#?](#)
7. [CompleteAdding method and IsCompleted Property of BlockingCollection<T> in C#](#)
8. [BlockingCollection in the Foreach loop](#)
9. [Working with Multiple Producers and Consumers using BlockingCollection in C#](#)
10. [BlockingCollection Features in C#](#)

What is BlockingCollection in C#?

The BlockingCollection is a Concurrent Collection Class in C# which provides thread safety. That means multiple threads can add as well as remove objects from the BlockingCollection concurrently.

The BlockingCollection implements the Producer-Consumer Pattern in C#. In Producer-Consumer Pattern, we have two threads one is called the Producer thread and another one is called the Consumer thread. And most important point is that both the threads will share a common collection class to exchange the data between them. And that scenario, we can use BlockingCollection as the collection class which will be shared by both Producer and Consumer threads. The Producer thread is going to generate the data while the consumer thread is going to consume the data. We can also set the maximum limit of the BlockingCollection collection class. And once we set the maximum limit of the collection, then the Producer cannot add new objects more than the maximum limit and the consumer cannot remove data from an empty collection class.

The BlockingCollection has two important features (these two features actually help us to implement Producer-Consumer Pattern) which differentiate it from other concurrent collection classes in C#. The two features are as follows:

1. **Bounding:** Bounding means as we already discussed we can set the maximum number of objects that we can store in the collection. When a producer thread reaches the BlockingCollection maximum limit, it is blocked to add new objects. In the blocked stage, the producer thread goes into sleep mode. It will unblock as soon as the consumer thread removes objects from the collection.
2. **Blocking:** Blocking means as we already discussed when the BlockingCollection is empty, the consumer thread is blocked until the producer thread adds new objects to the collections.

In the end, the producer thread will call the CompleteAdding() method of the BlockingCollection class. The CompleteAdding() method set the IsCompleted property to true. The consumer thread internally monitors the IsCompleted property to whether there are any items to consume from the collection. If this is not clear at the moment, then don't worry we will see everything with examples.

How to Create a BlockingCollection<T> instance in C#?

The BlockingCollection<T> Class in C# provides the following four constructors that we can use to create an instance of the BlockingCollection<T> class.

1. **BlockingCollection():** It initializes a new instance of the BlockingCollection class without an upper bound.
2. **BlockingCollection(int boundedCapacity):** It initializes a new instance of the BlockingCollection class with the specified upper bound. The boundedCapacity parameter specifies the bounded size of the collection. It will throw ArgumentOutOfRangeException if the bounded capacity is not a positive value.
3. **BlockingCollection(IProducerConsumerCollection<T> collection):** It initializes a new instance of the BlockingCollection class without an upper bound and uses the provided IProducerConsumerCollection as its underlying data store. Here, the parameter collection specifies the collection to use as the underlying data store. It will throw ArgumentNullException if the collection argument is null.
4. **BlockingCollection(IProducerConsumerCollection<T> collection, int boundedCapacity):** It initializes a new instance of the BlockingCollection class with the specified upper bound and uses the provided IProducerConsumerCollection as its underlying data store. Here, the boundedCapacity parameter specifies the bounded size of the collection. The parameter collection specifies the collection to use as the underlying data store. It will throw ArgumentNullException if the collection argument is null. It will throw ArgumentOutOfRangeException if the bounded capacity is not a positive value.

Let's see how to create an instance of BlockingCollection using the BlockingCollection() constructor:

Step1:

As the BlockingCollection<T> class belongs to System.Collections.Concurrent namespace, so first, we need to include System.Collections.Concurrent namespace in our program is as follows:

```
using System.Collections.Concurrent;
```

Step2:

Next, we need to create an instance of the BlockingCollection class using the BlockingCollection() constructor as follows:

```
BlockingCollection<type> BlockingCollection_Name = new BlockingCollection<type>();
```

Here, the type can be any built-in data type like int, double, string, etc., or any user-defined data type like Customer, Employee, Product, etc. As we have not set the max limit so it will take any number of items. For example,

```
BlockingCollection<int> blockingCollection = new BlockingCollection<int>();
```

In the below example, we have set the maximum limit to 10 so it will create the instance with the specified limit like 10.

```
BlockingCollection<int> blockingCollection = new BlockingCollection<int>(10);
```

Note: By default, the BlockingCollection uses ConcurrentQueue as its collection class. It is also possible to provide other concurrent collection classes like ConcurrentStack and ConcurrentBag. But, the most important point that you need to keep in mind is that here we can only pass those concurrent collection classes which implement the IProducerConsumerCollection<T> interface. And the ConcurrentStack<T> and ConcurrentBag<T> collection classes implement the IProducerConsumerCollection<T> interface. We can also define our own collection class which implements the IProducerConsumerCollection<T> interface and pass that class to the BlockingCollection constructor.

The following statement shows how to pass ConcurrentStack to the BlockingCollection constructor.

```
BlockingCollection<int> blockingCollection = new BlockingCollection<int>(new  
ConcurrentStack<int>());
```

Even, it is also possible to set the maximum limit as follows while passing ConcurrentStack to the BlockingCollection constructor.

```
BlockingCollection<int> blockingCollection = new BlockingCollection<int>(new  
ConcurrentStack<int>(), 10);
```

So, we have discussed the use of all four types of constructors to create an instance of BlockingCollection class in C#.

How to Add Elements into a BlockingCollection<T> in C#?

If you want to add elements to a BlockingCollection<T> in C#, then you need to use the following methods of the BlockingCollection<T> class.

1. **Add(T item):** This method is used to add the item to the BlockingCollection. Add method takes a single parameter i.e. the item to be added to the collection. The value can be null for a reference type. This method is blocked when the maximum limit is reached.

Following is the example of Add method.

```
BlockingCollection<int> blockingCollection = new BlockingCollection<int>(2);  
blockingCollection.Add(10);  
blockingCollection.Add(20);  
blockingCollection.Add(30);
```

In the above example, we have created BlockingCollection with the maximum capacity of 2 items. In this case, when we try to add the third item, it will block until an item is removed from the collection.

1. **TryAdd(T item):** This method tries to add the specified item to the BlockingCollection. The parameter item to be added to the collection. It returns true if the item could be added; otherwise, false. If the item is a duplicate, and the underlying collection does not accept duplicate items, then an InvalidOperationException is thrown.

Following is an example of the TryAdd method.

```
BlockingCollection<int> blockingCollection = new BlockingCollection<int>(2);  
blockingCollection.TryAdd(10);  
blockingCollection.TryAdd(20);  
blockingCollection.TryAdd(30);
```

We have a different TryAdd method with takes a timeout value as the second parameter. If the Tryadd operation is not completed within the timespan value then the TryAdd method returns with a false value. Following is the example.

```
BlockingCollection<int> blockingCollection = new BlockingCollection<int>(2);  
blockingCollection.Add(10);  
blockingCollection.Add(20);  
if (blockingCollection.TryAdd(30, TimeSpan.FromSeconds(1)))  
{  
    Console.WriteLine("Item 30 Added");  
}  
else  
{  
    Console.WriteLine("Item 30 Not added");  
}
```

```
}
```

In the above example, we have set the maximum capacity to 2 in the constructor. So, when we try to add the third item, it will wait for 1 second and returns with a false value.

How to access a BlockingCollection in C#?

We can access all the elements of the BlockingCollection in C# using a for each loop as follows.

```
foreach (var item in blockingCollection)
{
    Console.WriteLine(item);
}
```

Example to Understand How to Create a BlockingCollection and Add Elements in C#:

For a better understanding of how to create a BlockingCollection, how to add elements, and how to access all the elements from BlockingCollection in C# using a for-each loop, please have a look at the following example which shows the above three things.

```
using System;
using System.Collections.Concurrent;
namespace ConcurrentBagDemo
{
    class Program
    {
        static void Main()
        {
            // Creating an Instance of BlockingCollection Class with Capacity
            BlockingCollection<int> blockingCollection = new BlockingCollectio
            //Adding Element using Add Method
            blockingCollection.Add(10);
            blockingCollection.Add(20);
            //Adding Element using TryAdd Method
            blockingCollection.TryAdd(40);
            blockingCollection.TryAdd(50);
            if (blockingCollection.TryAdd(30, TimeSpan.FromSeconds(1)))
            {
                Console.WriteLine("Item 30 Added");
            }
            else
            {
                Console.WriteLine("Item 30 Not added");
            }
            //Accessing the BlockingCollection using For Each loop
            Console.WriteLine("\nAll BlockingCollection Elements");
            foreach (var item in blockingCollection)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

```

    }
}

```

Output:

```

Item 30 Not added

All BlockingCollection Elements
10
20
40
50

```

Initializing BlockingCollection in C# using Collection Initializer:

It is also possible in C# to Initializing a BlockingCollection using Collection Initializer as shown in the below example.

```

using System;
using System.Collections.Concurrent;
namespace ConcurrentBagDemo
{
    class Program
    {
        static void Main()
        {
            // Creating an Instance of BlockingCollection Class with Capacity
            BlockingCollection<int> blockingCollection = new BlockingCollectio
            {
                10,
                20,
                30,
                40,
                // 50 //It will block the blockingCollection as we set the capa
            };

            //Accessing the BlockingCollection using For Each loop
            Console.WriteLine("All BlockingCollection Elements");
            foreach (var item in blockingCollection)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}

```

Output:

```
All BlockingCollection Elements
10
20
30
40
```

Note: If you don't want to restrict the number of elements to be added to the collection, you just need to simply remove the capacity value from the constructor while creating an instance of BlockingCollection class in C#.

How to Remove Elements from BlockingCollection<T> Collection in C#?

The BlockingCollection<T> Class in C# provides the following methods to remove an element.

1. **Take():** This method is used to remove an item from the BlockingCollection. It returns the item removed from the collection. The Take method is blocked when the collection is empty. It'll unblock automatically when an item is added by another thread.
2. **TryTake(out T item):** This method tries to remove an item from the BlockingCollection. It will store the removed item in the output item parameter. It returns true if an item could be removed; otherwise, false.
3. **TryTake(out T item, TimeSpan timeout):** This method tries to remove an item from the BlockingCollection in the specified time period. The parameter timeout specifies an object that represents the number of milliseconds to wait or an object that represents -1 milliseconds to wait indefinitely. It returns true if an item could be removed from the collection within the specified time; otherwise, false. If the collection is empty then this method will wait for the time specified in the timeout parameter. If the new item is not added within the timeout value, then it returns false.
4. **TryTake(out T item, int millisecondsTimeout):** This method tries to remove an item from the System.Collections.Concurrent.BlockingCollection in the specified time period. The parameter millisecondsTimeout specifies the number of milliseconds to wait or System.Threading.Timeout.Infinite (-1) to wait indefinitely. It returns true if an item could be removed from the collection within the specified time; otherwise, false. If the collection is empty then this method will wait for the time specified in the timeout parameter. If the new item is not added within the timeout value, then it returns false.

Let us see an example to understand the above methods of BlockingCollection<T> Class in C#. Please have a look at the following example which shows the use of all the above take and TryTake methods.

```
using System;
using System.Collections.Concurrent;
namespace ConcurrentBagDemo
{
    class Program
    {
        static void Main()
        {
            // Creating an Instance of BlockingCollection Class without Capacity
            BlockingCollection<int> blockingCollection = new BlockingCollection<int>
            {
                10,
                20
            };
        }
    }
}
```

```

//Accessing the BlockingCollection using For Each loop
Console.WriteLine("All BlockingCollection Elements");
foreach (var item in blockingCollection)
{
    Console.WriteLine(item);
}
//Removing item using Take Method
int Result1 = blockingCollection.Take();
Console.WriteLine($"Item Removed By Take Method: {Result1}");
//Removing item using TryTake Method
if (blockingCollection.TryTake(out int Result2, TimeSpan.FromSeconds(1))
{
    Console.WriteLine($"Item Removed By TryTake Method: {Result2}");
}
else
{
    Console.WriteLine("No Item Removed By TryTake Method");
}
//No More Elements in the Collections and Trying to Remove Item using
Method
if (blockingCollection.TryTake(out int Result3, TimeSpan.FromSeconds(1))
{
    Console.WriteLine($"Item Removed By TryTake Method: {Result3}");
}
else
{
    Console.WriteLine("No Item Removed By TryTake Method");
}
Console.ReadKey();
}
}
}

```

Output:

```

All BlockingCollection Elements
10
20

Item Removed By Take Method: 10

Item Removed By TryTake Method: 20

No Item Removed By TryTake Method

```

Complete Adding method and IsCompleted Property of BlockingCollection<T> in C#:

The Producer thread calls the CompleteAdding method. The CompleteAdding method internally marks the IsAddingCompleted property to true. The IsCompleted property is used by the consumer threads. It returns true when IsAddingCompleted is true and the BlockingCollection is empty. That means when IsCompleted is true there are no items in the collection and other producer threads will not add any new item.

1. **CompleteAdding():** The CompleteAdding method Marks the BlockingCollection instances as not accepting any more additions.
2. **IsAddingCompleted { get; }:** This property returns true if the BlockingCollection has been marked as complete for adding else it will return false.
3. **IsCompleted { get; }:** This property returns true if the BlockingCollection has been marked as complete for adding and is empty else it will return false.

Let us understand the above CompleteAdding Method and IsAddingCompleted and IsCompleted properties with an example. For a better understanding, please have a look at the below example. In the below example, we have created two threads i.e. producerThread and consumerThread. The producerThread will add items into the BlockingCollection. After adding all the required items, it calls the CompleteAdding method which will mark the collection class to not add any more items. The consumerThread put a condition in the while loop. In the loop, it checks the IsCompleted property. The while loop will run as long as the IsCompleted property returns false. Then, from the BlockingCollection, we remove one item at a time using the Take method and print that item on the console window.

```
using System;
using System.Collections.Concurrent;
using System.Threading.Tasks;
namespace ConcurrentBagDemo
{
    class Program
    {
        static void Main()
        {
            BlockingCollection<int> blockingCollection = new BlockingCollection<int>();
            //Thread 1 (Producer Thread) Adding Item to blockingCollection
            Task producerThread = Task.Factory.StartNew(() =>
            {
                for (int i = 0; i < 10; ++i)
                {
                    blockingCollection.Add(i);
                }
                //Mark blockingCollection will not accept any more additions
                blockingCollection.CompleteAdding();
            });
            //Thread 2 (Consumer Thread) Removing Item from blockingCollection
            //Printing on the Console
            Task consumerThread = Task.Factory.StartNew(() =>
            {
                //Loop will continue as long as IsCompleted returns false
                while (!blockingCollection.IsCompleted)
                {
                    int item = blockingCollection.Take();
                    Console.WriteLine($"{item} ");
                }
            });
        }
    }
}
```

```

        Task.WaitAll(producerThread, consumerThread);
        Console.ReadKey();
    }
}
}

```

Output: 0 1 2 3 4 5 6 7 8 9

BlockingCollection in the Foreach loop:

The `BlockingCollection<T>` Class in C# provides the `GetConsumingEnumerable()` method.

1. **`IEnumerable<T> GetConsumingEnumerable()`**: This method returns `IEnumerable<T>` so that we can use that method in the foreach loop. This method returns items as soon as items are available in the collection. The `GetConsumingEnumerable()` method has a blocking feature. It will block the foreach loop when the collection is empty. A foreach loop ends when the producer thread calls the `CompleteAdding` method.

For a better understanding, please have a look at the below example. In the below example, the producer thread is adding items to the `BlockingCollection`. It will sleep for 1 second before adding items to the collection. The `GetConsumingEnumerable` method waits until the `CompleteAdded` method is called.

```

using System;
using System.Collections.Concurrent;
using System.Threading;
using System.Threading.Tasks;
namespace ConcurrentBagDemo
{
    class Program
    {
        static void Main()
        {
            BlockingCollection<int> blockingCollection = new BlockingCollection<int>();
            //Thread 1 (Producer Thread) Adding Item to blockingCollection
            Task producerThread = Task.Factory.StartNew(() =>
            {
                for (int i = 0; i < 10; ++i)
                {
                    Thread.Sleep(TimeSpan.FromSeconds(1));
                    blockingCollection.Add(i);
                }
                //Mark blockingCollection will not accept any more additions
                blockingCollection.CompleteAdding();
            });
            foreach (int item in blockingCollection.GetConsumingEnumerable())
            {
                Console.Write($"{item} ");
            }
            Console.ReadKey();
        }
    }
}

```



```

        {
            producers[0].Add(i);
            Thread.Sleep(100);
        }
        producers[0].CompleteAdding();
    });
    Task t2 = Task.Factory.StartNew(() =>
    {
        for (int i = 11; i <= 20; ++i)
        {
            producers[1].Add(i);
            Thread.Sleep(150);
        }
        producers[1].CompleteAdding();
    });
    Task t3 = Task.Factory.StartNew(() =>
    {
        for (int i = 21; i <= 30; ++i)
        {
            producers[2].Add(i);
            Thread.Sleep(250);
        }
        producers[2].CompleteAdding();
    });
    while (!producers[0].IsCompleted || !producers[1].IsCompleted ||
!producers[2].IsCompleted)
    {
        BlockingCollection<int>.TryTakeFromAny(producers, out int item);
        TimeSpan.FromSeconds(1));
        if (item != default(int))
        {
            Console.WriteLine($"{ item} ");
        }
    }
    Console.ReadKey();
}
}
}

```

Output:

```
1 11 21 2 12 3 22 13 4 5 14 23 6 15 7 8 16 24 9 17 10 25 18 19 26 20 27 28 29 30
```

BlockingCollection Features in C#:

BlockingCollection<T> is a thread-safe collection class that provides the following features:

1. An implementation of the Producer-Consumer pattern.
2. Concurrent adding and taking of items from multiple threads.

3. Optional maximum capacity.
4. Insertion and removal operations block when the collection is empty or full.
5. Insertion and removal “try” operations that do not block or that block up to a specified period of time.
6. Encapsulates any collection type that implements `IProducerConsumerCollection<T>`

In the next article, I am going to discuss [File Handling in C#](#) with Examples. Here, in this article, I try to explain the **BlockingCollection** **Collection in C#** with Examples. I hope this BlockingCollection Class in C# with Examples article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.